Lecture 11: Neural networks

Machine Learning 2025

Federico Chiariotti (federico.chiariotti@unipd.it)



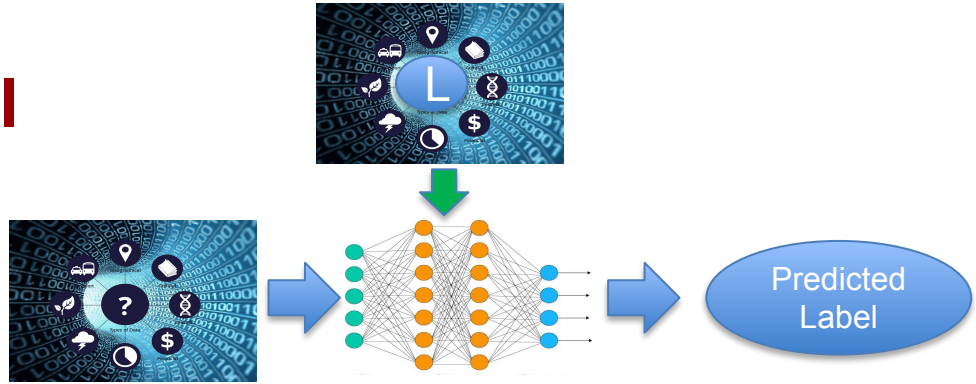
Università degli Studi di Padova

Recap



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Supervised learning model



- Prediction rule $h : X \rightarrow Y$
  - The learner's output (hypothesis)
  - $A(S)$: hypothesis produced by algorithm A when it is fed training set S
- Data generation model
  - D is a distribution over X **(unknown to the machine learning algorithm)**
  - Instances are labeled according to $f : X \rightarrow Y$ **(unknown to the ML algorithm)**
  - Training set: sampling according to $D, y_i = f(x_i)\ \forall x_i \in S$
- Success metric
  - Classifier error: probability of predicting the wrong label over D

Empirical Risk Minimization (ERM)

- ➔ Learner outputs $h_S : X \to Y$
- ➔ **Goal: find $\mathbf{h}^*$ that minimizes $\mathbf{L_{D,f}(h)}$**
- ➔ Both D and f are unknown!

ERM: we minimize the loss on the training set $L_S(h) = \frac{\sum_{i=1}^{m} |h(x_i) - y_i|}{m}$

This works if we use a 0-1 loss: the definition can be generalized

The empirical risk is also called training error or training loss

No free lunch theorem

Let A be any learning algorithm for binary classification, with 0-1 loss, over a domain X. Let m be any number smaller than $\frac{|X|}{2}$. There exists a distribution D over $X \times \{0,1\}$ such that:

1. There exists a function $f : X \to \{0,1\}$ with $L_D(f) = 0$
2. We have $L_D(A(S) \geq \frac{1}{8}$ with probability $\frac{1}{7}$ over the choice of $S \sim D^m$

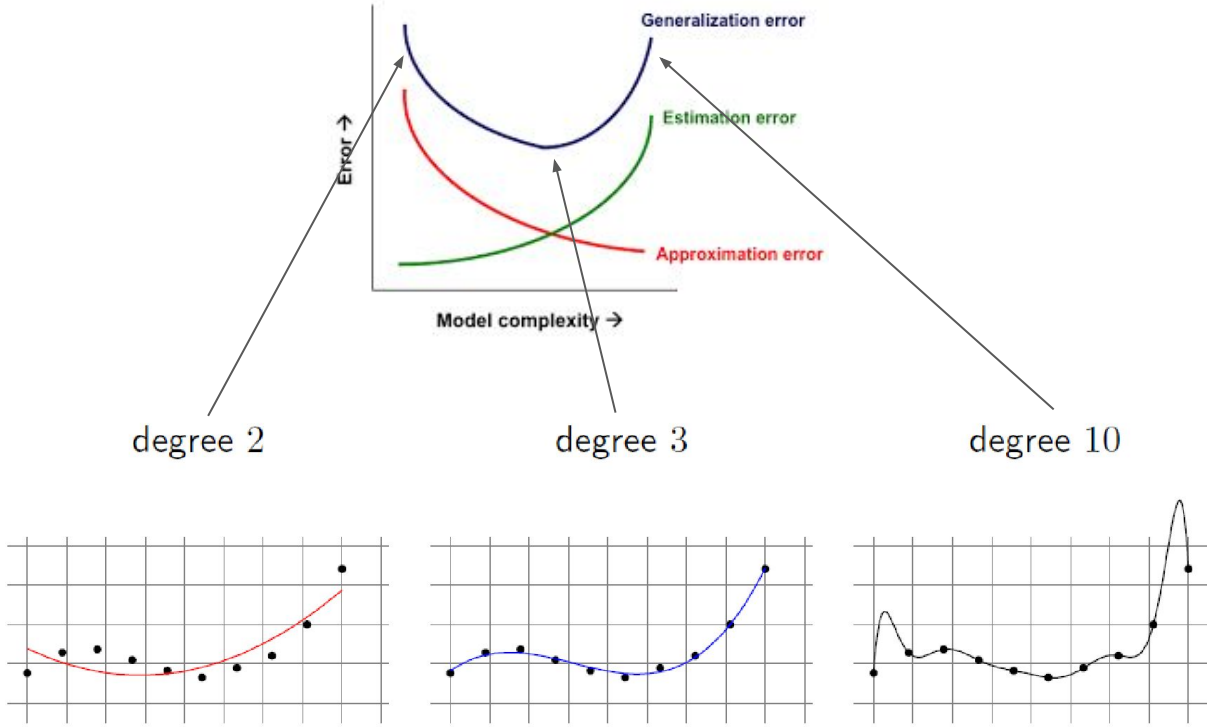
Corollary: if H is the set of all functions and X is an infinite set, we do **not** have PAC learnability

The fundamental theorem of PAC learning

Let H be a hypothesis class for a binary classification problem with 0-1 loss. The following statements are equivalent:

1. H has the uniform convergence property
2. Any ERM rule is a successful agnostic PAC learner for H
3. H is agnostic PAC learnable
4. H is PAC learnable (under realizability condition)
5. Any ERM rule is a successful PAC learner for H (under realizability condition)
6. H has a finite VC dimension

Example: polynomial fitting



Convex programming

A problem is convex if we have two conditions:

- The objective function is convex
- The feasible set is convex
  - Inequality constraints should be convex
  - Equality constraints should be affine functions

We can solve convex problems efficiently!

Primal and dual problems

We can define a Lagrangian dual problem by turning a constrained problem into an unconstrained one.

Each constraint becomes a penalty in the objective function, with an associated Lagrange multiplier

The solution to the dual problem is *better* than the solution to the primal problem (but it might not be valid)



Example: primal and dual problems

Problem: minimize $f(x, y) = (x + y)^2$ such that $x^2 + y^2 \leq 1$

We move to the unconstrained problem

$$f'(x, y) = (x + y)^2 + \lambda(x^2 + y^2 - 1)$$

This is still a convex function! We can find the minimum by setting the gradient to 0. By gradually increasing the multiplier, we get closer and closer to the primal problem

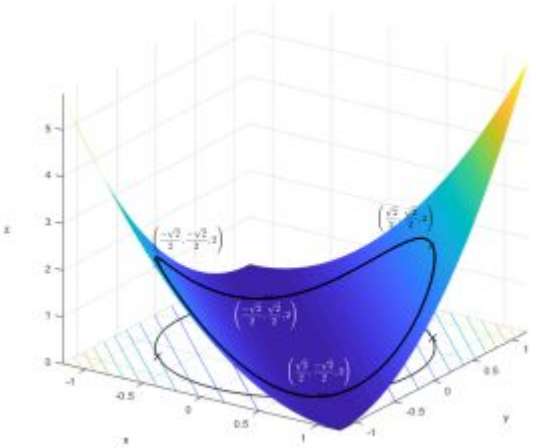


Image: Wikimedia Foundation

Using the gradient

Since we have a single minimum, how about trying to follow the gradient?

This is something like a ball falling down a curved surface and coming to rest on the lowest point

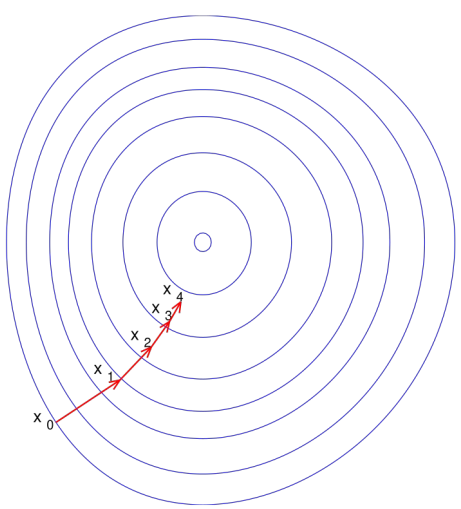


Image: Wikimedia Foundation

Gradient descent

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_d} \right)$$

The gradient points towards the direction of steepest ascent, it corresponds to the first-order Taylor approximation of the function

The approximation works for small steps, but breaks down over larger ones

Gradient descent algorithm

GD algorithm:

$\boldsymbol{w}^{(0)} \leftarrow \mathbf{0}$

for $t \leftarrow 0$ to $T - 1$ do

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla f(\boldsymbol{w}^{(t)})$$

return $\bar{\boldsymbol{w}} = \boldsymbol{w}^{(T)}$

Learning rate
(controls the size of the steps)

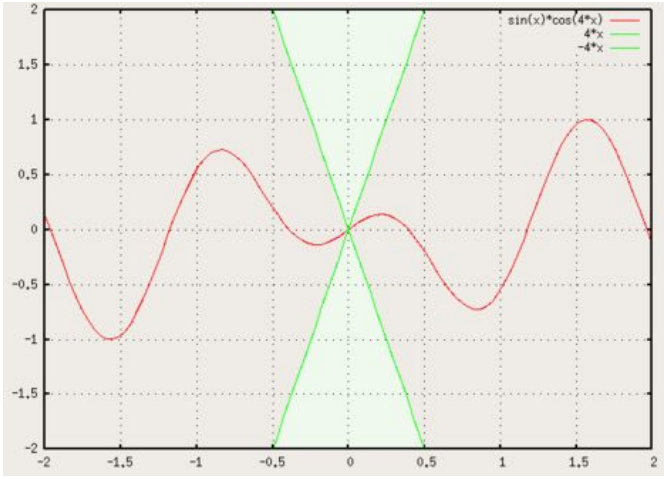
We stop when the gradient is 0 (minimum) or when we reach a maximum number of iterations

Gradient descent: convergence

If the target function is convex and ρ-Lipschitz, and we run GD for T steps, we have:

$$\eta = \sqrt{\frac{B^2}{\rho^2 T}} \Rightarrow f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \frac{B\rho}{\sqrt{T}}$$

Upper bound on $||\mathbf{w}^*||$



Proof: not part of the course (see Sec. 14.1.1 in the book)

A simple corollary

If the target function is convex and ρ-Lipschitz, and we run GD for T steps, we have:

$$\eta = \sqrt{\frac{B^2}{\rho^2 T}} \Rightarrow f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \frac{B\rho}{\sqrt{T}}$$

If we want a precision ε, we can run the GD algorithm over enough steps.

$$T \geq \frac{B^2\rho^2}{\varepsilon^2} \Rightarrow f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \varepsilon$$

This is not learning: the error bound is with probability 1!

Ensemble models

Ensemble models rely on a simple principle: different learners may capture different aspects of the problem, and combining multiple learners may increase their power

If we can add some randomness to individual model, we can avoid overfitting and reduce computational complexity: multiple simple model can be more robust than a single complex one!

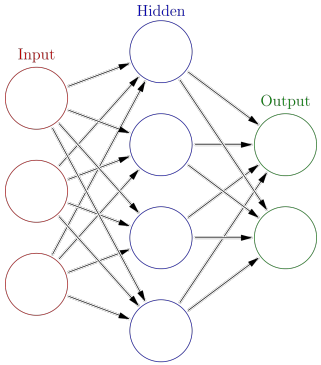
Part 1:
Neural networks

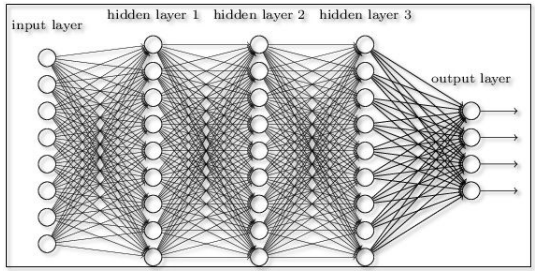


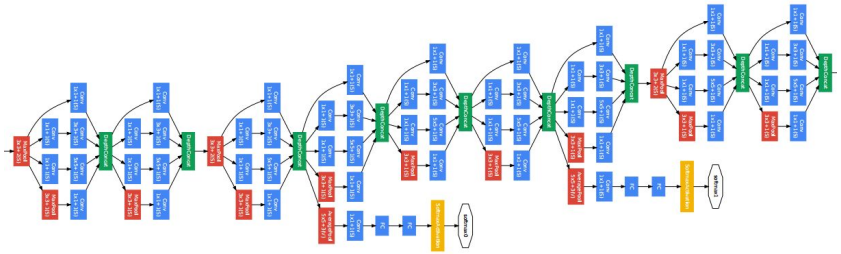
UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Artificial neural networks

Neural networks are inspired by biological neurons, like the perceptron. Although the idea has been around since the 1980s, practical results surpassed SVMs only in the 2000s. The past 15-20 years have seen a real deep neural network boom, with impressive performance gains







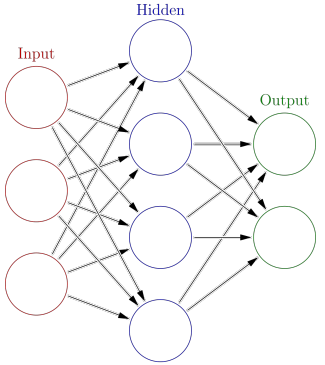
Neural network properties

Deep neural networks are extremely powerful models, with a high risk of overfitting

➔ Very high risk of overfitting
➔ Almost no prior knowledge encoded in the data (see: physics-informed NNs)
➔ Need for much larger volumes of data

More complex neural network models, such as convolutional NNs (CNNs) or large language models (LLMs), need huge volumes of data to be properly trained

Neural network model



Weighted directed graph $G = (V, E, \mathbf{w})$

- ➔ V is the set of neurons
- ➔ $E \subseteq V \times V$ is the set of connections
- ➔ $\mathbf{w} : E \to \mathbb{R}$ is the weight function (the weights are the parameters to be learned!)

For each neuron v:

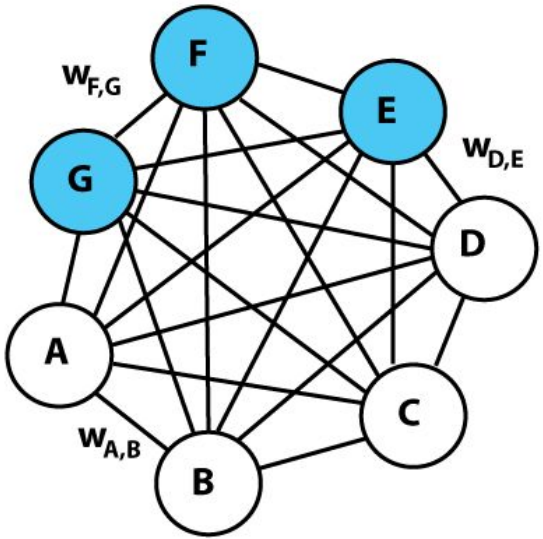
1. The input i_v is the sum of the outputs of the incoming connections, weighted by their edge weight: $i_v = \sum_{u \in V:(u,v) \in E} w_{(u,v)} o_u$
2. The output o_v is the result of an activation function $\sigma(i_v)$

Boltzmann machines

Boltzmann machines are spin-glass models: visible units (in blue) have fixed values (inputs from the environments), while hidden units are updated following energy equations.

The weights of the connections are gradually adapted to minimize the energy, i.e., maximize the likelihood of the inputs

Boltzmann machines (and energy-based models) are used in modern generative AI, but are too complex for straightforward learning tasks

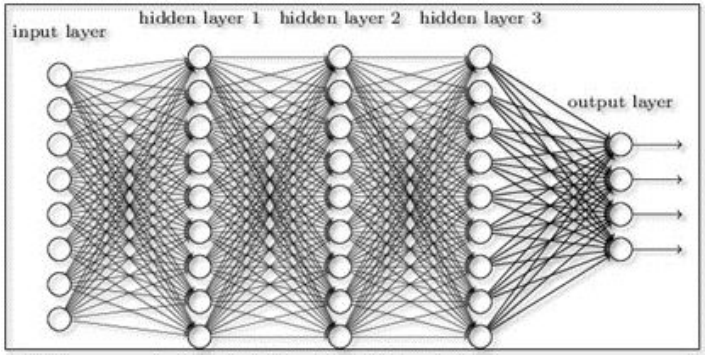


Feedforward neural networks

We can impose a structure to simplify training: the network is organized in **layers**, and each layer only takes the outputs of the previous layer as inputs

We then partition the neurons: $V = \bigcup_{t=0}^{T} V_t, \quad V_t \cap V_{t'} = \varnothing \; \forall t \neq t'$

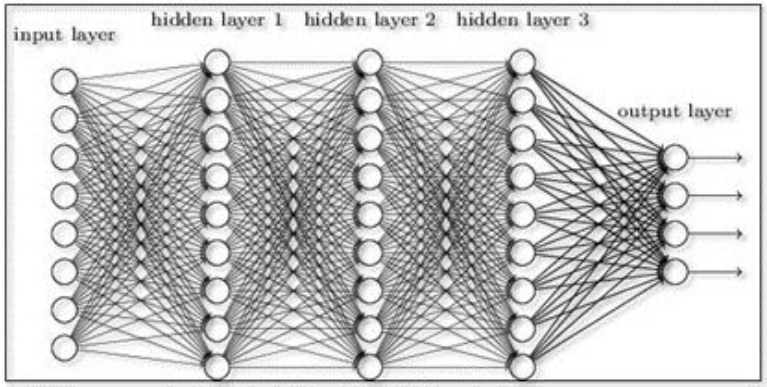
We have $(u, v) \in E \implies \exists t : u \in V_t \wedge v \in V_{t+1}$: each neuron is connected only to neurons at the next layer



Feedforward neural networks

We consider **fully connected** feedforward NNs: each neuron of layer t is connected to all neurons of layer t+1 (with T≥2)

➔ Input for layer 0: the input to our learning problem
➔ Output of layer T: the predicted label
➔ In each layer, we introduce a bias neuron with input 1



The weight matrix

Each layer (except the first) has a matrix $\mathbf{w}^{(t)}$, with size $|V_{t-1}| \times |V_t - 1|$

$$\mathbf{w}^{(t)} = \begin{pmatrix} w_{01}^{(t)} & w_{02}^{(t)} & \cdots & w_{0|V_t|}^{(t)} \\ w_{11}^{(t)} & w_{12}^{(t)} & \cdots & w_{1|V_t|}^{(t)} \\ \vdots & \vdots & \vdots & \vdots \\ w_{|V_{t-1}|1}^{(t)} & w_{|V_{t-1}|2}^{(t)} & \cdots & w_{|V_{t-1}||V_t|}^{(t)} \end{pmatrix}$$

Note that neuron 0 of each layer has no input weights (it is the bias!)

How many weights are there?

The total number of learnable parameters (i.e., weights) of a neural network is

$$\sum_{t=1}^{T-1} |V_{t-1}|(|V_t| - 1) + |V_{T-1}||V_T|$$

Remove the bias neuron

Output layer: no bias neuron

The number of inputs (including the bias) is $|V_0|$, the number of outputs is $|V_t|$

Example: counting weights

Let us consider a classification task. The input is a 4x4 grayscale image, and there are 4 classes. The network has 2 hidden layers: the first has 10 neurons, while the second has 6 (both excluding the bias)

Example: counting weights

Let us consider a classification task. The input is a 4x4 grayscale image, and there are 4 classes. The network has 2 hidden layers: the first has 10 neurons, while the second has 6 (both excluding the bias)

1. There are 16 inputs, plus the bias. The first layer has 10 neurons, so the first weight matrix's dimension is 17×10

Example: counting weights

Let us consider a classification task. The input is a 4x4 grayscale image, and there are 4 classes. The network has 2 hidden layers: the first has 10 neurons, while the second has 6 (both excluding the bias)

1. There are 16 inputs, plus the bias. The first layer has 10 neurons, so the first weight matrix's dimension is 17×10
2. Considering 11 inputs (10 + bias) and 6 outputs, the second matrix's dimension is 11×6

Example: counting weights

Let us consider a classification task. The input is a 4x4 grayscale image, and there are 4 classes. The network has 2 hidden layers: the first has 10 neurons, while the second has 6 (both excluding the bias)

1. There are 16 inputs, plus the bias. The first layer has 10 neurons, so the first weight matrix's dimension is 17×10
2. Considering 11 inputs (10 + bias) and 6 outputs, the second matrix's dimension is 11×6
3. Finally, we have 4 outputs and 7 (6 + bias) inputs: the third matrix's dimension is 7×4

Example: counting weights

Let us consider a classification task. The input is a 4x4 grayscale image, and there are 4 classes. The network has 2 hidden layers: the first has 10 neurons, while the second has 6 (both excluding the bias)

1. There are 16 inputs, plus the bias. The first layer has 10 neurons, so the first weight matrix's dimension is 17×10
2. Considering 11 inputs (10 + bias) and 6 outputs, the second matrix's dimension is 11×6
3. Finally, we have 4 outputs and 7 (6 + bias) inputs: the third matrix's dimension is 7×4

$$17 \times 10 + 11 \times 6 + 7 \times 4 = 170 + 66 + 28 = \boxed{264}$$

Neurons

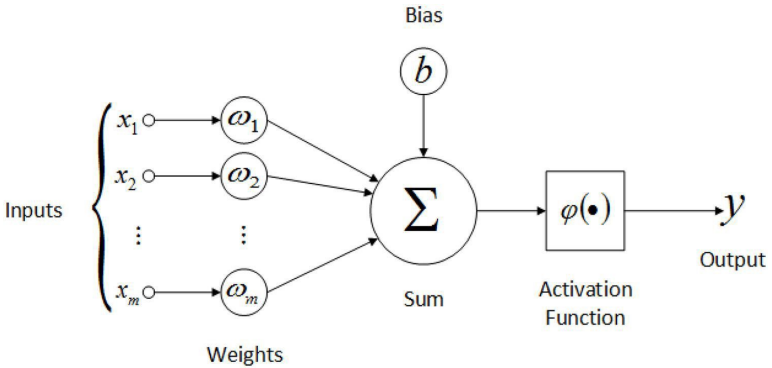
The output $o_{t,i}(\mathrm{x})$ of the i-th neuron in the t-th layer is computed recursively. We also use $\mathbf{v}^{(t)}$ to indicate the output of the whole layer in a compact form.

The input to the neuron is $a_{t+1,j} = \langle \mathbf{w}_j^{(t+1)}, \mathbf{v}^{(t)} \rangle$

The output of the neuron is $o_{t+1,j} = \sigma(a_{t+1,j})$

$$\mathbf{w}^{(t)} = \begin{pmatrix} w_{01}^{(t)} & w_{02}^{(t)} & \cdots & w_{0|V_t|}^{(t)} \\ w_{11}^{(t)} & w_{12}^{(t)} & \cdots & w_{1|V_t|}^{(t)} \\ \vdots & \vdots & \vdots & \vdots \\ w_{|V_{t-1}|1}^{(t)} & w_{|V_{t-1}|2}^{(t)} & \cdots & w_{|V_{t-1}||V_t|}^{(t)} \end{pmatrix}$$

$\mathbf{w}_2^{(t)}$



Neurons

Computing the input of the first layer is simple:

$$a_{1,j} = w_{0,j}^{(1)} + \sum_{i=1}^{\ell(\mathbf{x})} w_{i,j}^{(1)} x_i$$

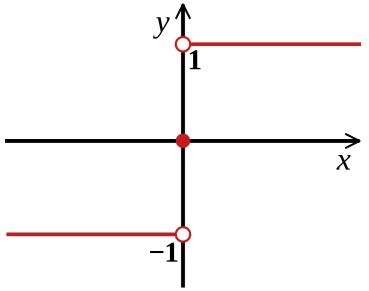
The following layers work recursively:

$$a_{t,j} = w_{0,j}^{(t)} + \sum_{i=1}^{|V_{t-1}|-1} w_{i,j}^{(t)} o_{t-1,i}$$

Activation functions: threshold

Each neuron is a perceptron

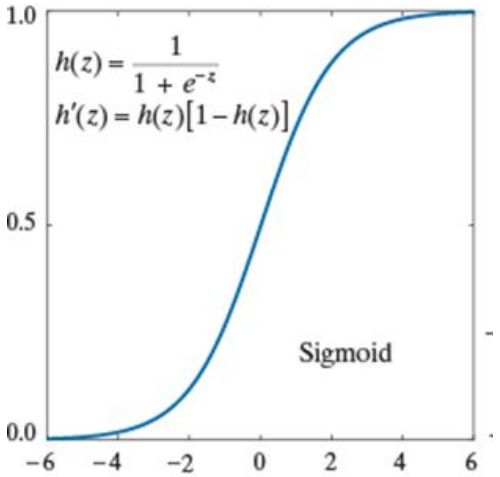
- ➔ Computationally simple
- ➔ Easy biological analogy (firing rate)
- ➔ Discontinuity in the output: gradients are either 0 or infinite



Activation functions: sigmoid

Soft version of the threshold (like in logistic regression)

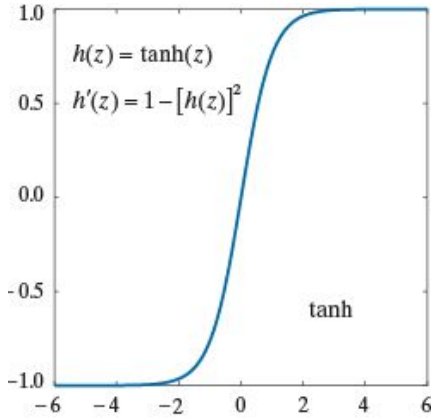
- ➔ Smooth output
- ➔ Easy biological analogy (firing rate)
- ➔ Saturation: high outputs have a very low gradient
- ➔ Computationally complex



Activation functions: hyperbolic tangent

Scaled and shifted sigmoid function

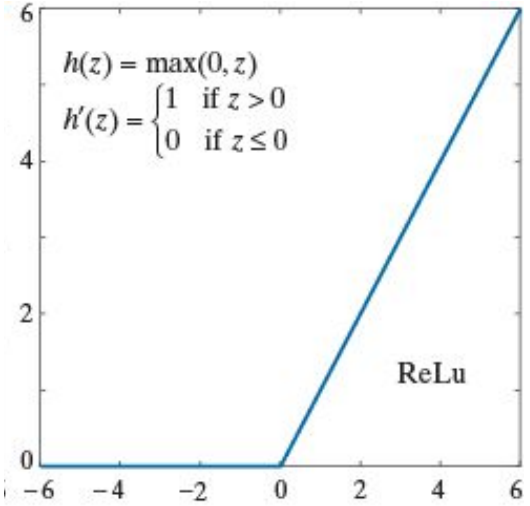
- ➔ Smooth output
- ➔ Easy biological analogy (firing rate)
- ➔ Zero-centered
- ➔ Saturation: high outputs have a very low gradient
- ➔ Computationally complex



Activation functions: rectilinear unit

ReLU: most commonly used activation function

➔ (Almost) smooth output
➔ Computationally simple
➔ Gradients do not vanish



Putting it all together: forward propagation

Input: $\mathbf{x} = (x_1, \ldots, x_d)^T$; NN with 1 output node
Output: prediction y of NN;

$v^{(0)} \leftarrow (1, x_1, \ldots, x_d)^T$;
for $t \leftarrow 1$ **to** T **do**
    $a^{(t)} \leftarrow \left(w^{(t)}\right)^T v^{(t-1)}$;
    $v^{(t)} \leftarrow \left(1, \sigma\left(a^{(t)}\right)^T\right)^T$;
$y \leftarrow v^{(T)}$;
return y;

Neural networks as hypothesis classes

A network (i.e., a hypothesis) is characterized by $V, E, \sigma, \mathbf{w}$

$$h_{V,E,\sigma,\mathbf{w}} : \mathbb{R}^{|V_0|-1} \rightarrow \mathbb{R}^{|V_T|}$$

We define the hypothesis class by setting the architecture V, E, σ

$$H_{V,E,\sigma} = \left\{ h_{V,E,\sigma,\mathbf{w}} \forall \mathbf{w} \in \mathbb{R}^{|E|} \right\}$$

The expressive power of NNs

For every dimension d, there is a feedforward graph of depth 2 (V, E) such that the corresponding class $H_{V,E,\text{sign}}$ contains all possible functions mapping $\{-1, 1\}^d$ to $\{-1, 1\}$. The minimum size $s(d)$ of such a class is an exponential function of d.

The proof of this is simple: if we consider a hidden layer with $2^d + 1$ neurons, each neuron is mapped to one possible input. We then set a weight 0 for all neurons whose input belongs to class -1, weight -1 to the bias, and weight +1 to all neurons whose corresponding input belongs to class +1.

This overfits a lot!

Universal function approximation

We can represent real numbers as bit strings (and computers always do!), to get a universal approximator (at least in practice).

Neural networks can actually implement any function with arbitrary precision (as proven by Cybenko in 1989), but require an exponential size.

Naturally, we do **not** want a universal approximator, to avoid ending up in no free lunch territory!

The VC dimension of NNs

With sign activation (perceptron-like neurons):

$$VC(H_{V,E,\text{sign}}) = O(|E| \log(|E|)$$

With sigmoid activation:

$$VC(H_{V,E,\text{sigmoid}}) = O(|V|^2|E|^2)$$

NNs require **a lot** of data with respect to classical ML! Additionally, we can prove that ERM over neural networks is an NP-hard problem: there is no known solution that works in polynomial time

A practical learning algorithm

Target of ERM: minimize the empirical risk

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^{m} \ell(y_i, h_{\mathbf{w}}(\mathbf{x}_i))$$

We can use SGD to follow the gradients and approximate a solution, as the problem cannot be solved in polynomial time

1. Compute the predicted output
2. **Compute the gradient**
3. Perform SGD

The gradient is with respect to all weights, but the output is after the last layer!

Backpropagation: last layer

Let us start from the last layer (considering a batch of B samples):

$$\frac{\partial L_S}{\partial \mathbf{w}^{(T)}} = \frac{\partial}{\partial \mathbf{w}^{(T)}} \frac{1}{B} \sum_{i=1}^{B} \ell(y_i, \mathbf{o}_i^{(T-1)} \mathbf{w}^{(T)})$$

We can apply the chain rule $\frac{df(g(x))}{dx} = \frac{df(g(x))}{dg(x)} \frac{df(x)}{dx}$ to get:

$$\frac{\partial L_S}{\partial \mathbf{w}^{(T)}} = \frac{1}{B} \sum_{i=1}^{B} \frac{\partial \ell(y_i, \hat{y}(\mathbf{x}_i))}{\partial \hat{y}(\mathbf{x}_i))} \mathbf{o}_i^{(T-1)}$$

Derivative of a linear function

Backpropagation: definitions

For each layer, we define $\delta^{(t)} = \dfrac{\partial L_S}{\partial \mathbf{a}^{(t)}}$

For the last layer, we have $\delta^{(T)} = \dfrac{1}{B}\sum_{i=1}^{B} \dfrac{\partial \ell(y_i, \hat{y}(\mathbf{x}_i))}{\partial \hat{y}(\mathbf{x}_i))}$

We rewrite the last layer gradient as:

$$\frac{\partial L_S}{\partial \mathbf{w}^{(T)}} = \delta^{(T)} \frac{1}{B} \sum_{i=1}^{B} \mathbf{o}_i^{(T-1)}$$

Backpropagation: next step

For each layer, we define $\delta^{(t)} = \dfrac{\partial L_S}{\partial \mathbf{a}^{(t)}}$

For the last layer, we have $\delta^{(T)} = \dfrac{1}{B}\sum_{i=1}^{B} \dfrac{\partial \ell(y_i, \hat{y}(\mathbf{x}_i))}{\partial \hat{y}(\mathbf{x}_i))}$

We rewrite the last layer gradient as:

$$\frac{\partial L_S}{\partial \mathbf{w}^{(T)}} = \delta^{(T)} \frac{1}{B} \sum_{i=1}^{B} \mathbf{o}_i^{(T-1)}$$

We also note that $\mathbf{o}_i^{(t)} = \sigma(\mathbf{a}_i^{(t)})$: we can extend this to a generic layer

Backpropagation: generic layer

$$\frac{\partial L_S}{\partial w_{ij}^{(t)}} = \frac{\partial L_S}{\partial a_j^{(t)}} \frac{\partial a_j^{(t)}}{\partial w_{ij}^{(t)}}$$

$\delta_j^{(t)}$

We substitute the linear function:

$$\frac{\partial L_S}{\partial w_{ij}^{(t)}} = \delta_j^{(t)} o_i^{(t-1)}$$

$$a_j^{(t)} = \sum_{i=0}^{|V_{t-1}|-1} w_{ij}^{(t)} o_i^{(t-1)}$$

We can also compute $\delta_j^{(t)}$, as it depends on the activation function:

$$\delta_j^{(t)} = \frac{\partial L_S}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial a_j^{(t)}} = \frac{\partial L_S}{\partial o_j^{(t)}} \sigma'(a_j^{(t)})$$

Backpropagation: generic layer

$$\delta_j^{(t)} = \frac{\partial L_S}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial a_j^{(t)}} = \frac{\partial L_S}{\partial o_j^{(t)}} \sigma'(a_j^{(t)})$$

We can compute this term by applying the chain rule on the next layer:

Missing term

$$\frac{\partial L_S}{\partial o_j^{(t)}} = \sum_{k=1}^{|V_{t+1}|} \frac{\partial L_S}{\partial a_k^{(t+1)}} \frac{\partial a_k^{(t+1)}}{\partial o_j^{(t)}}$$

$\delta_k^{(t+1)}$

$$a_k^{(t+1)} = \sum_{j=0}^{|V_t|-1} w_{jk}^{(t+1)} o_j^{(t)}$$

$$\frac{\partial L_S}{\partial o_j^{(t)}} = \sum_{k=1}^{|V_{t+1}|-1} w_{jk}^{(t+1)} \delta_k^{(t+1)}$$

Backpropagation: putting it all together

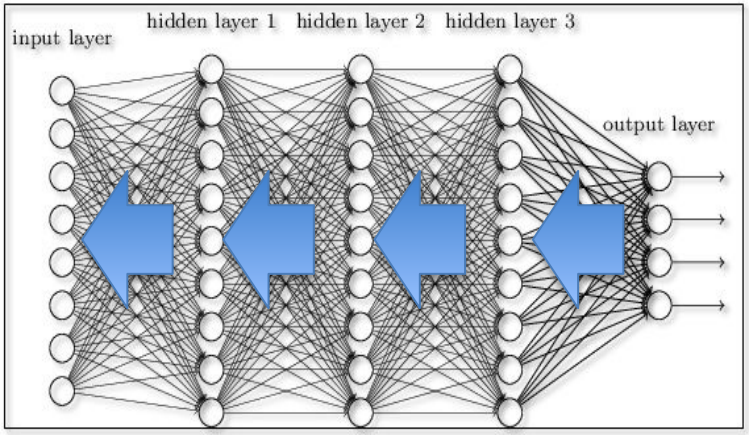
$$\delta_j^{(t)} = \frac{\partial L_S}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial a_j^{(t)}} = \frac{\partial L_S}{\partial o_j^{(t)}} \sigma'(a_j^{(t)})$$

$$\frac{\partial L_S}{\partial o_j^{(t)}} = \sum_{k=1}^{|V_{t+1}|-1} w_{jk}^{(t+1)} \delta_k^{(t+1)}$$

We can compute gradients recursively:

➔ Base step $\delta^{(T)} = \frac{1}{B} \sum_{i=1}^{B} \frac{\partial \ell(y_i, \hat{y}(\mathbf{x}_i))}{\partial \hat{y}(\mathbf{x}_i))}$

➔ Recursive update $\delta_j^{(t)} = \sigma'(a_j^{(t)}) \sum_{k=1}^{|V_{t+1}|-1} w_{jk}^{(t+1)} \delta_k^{(t+1)}$



Backpropagation pseudocode

Input: data point $(\mathbf{x}_i, y_i)$, NN (with weights $w_{ij}^{(t)}$, for $1 \leq t \leq T$)

Output: $\delta^{(t)}$ for $t = 1, \ldots, T$

compute $a^{(t)}$ and $v^{(t)}$ for $t = 1, \ldots T$;

$\delta^{(L)} \leftarrow \frac{\partial L}{\partial a^{(L)}}$;

for $t = T - 1$ **downto** 1 **do**

  $\delta_j^{(\ell)} \leftarrow \sigma'(a_{t,j}) \cdot \sum_{k=1}^{d^{(\ell+1)}} w_{jk}^{(t+1)} \delta_k^{(t+1)}$ for all $j = 1, \ldots, d^{(t)}$;

return $\delta^{(1)}, \ldots, \delta^{(T)}$;

SGD with backpropagation

BackPropagation algorithm with SGD

Input: training data $(x_1, y_1), \ldots, (x_m, y_m)$

Output: NN weights $w_{ij}^{(t)}$

Initialize $w_{ij}^{(t)} \quad \forall i, j, t$;

for $s \leftarrow 0,1,2, \ldots$ **do** // until convergence

pick (x_k, y_k) at random from training data; // SGD: 1 sample at random

compute $v_{t,j} \quad \forall j, t$; // forward propagation

compute $\delta_j^{(t)} \quad \forall j, t$; // backward propagation

$w_{ij}^{(t)[s+1]} = w_{ij}^{(t)[s]} - \eta v_{t-1,i} \delta_j^{(t)} \quad \forall i, j, t$; // update weights

if *converged* **then return** $w_{ij}^{(t)[s]} \quad \forall i, j, t$;

Regularization

We can include the L1 or L2 norm of the weights with the loss (the gradient is simple!)

$$L_S(h) + \frac{\lambda}{m}||\mathbf{w}||^2$$

The weighting parameter can be optimized with K-fold

Practical considerations

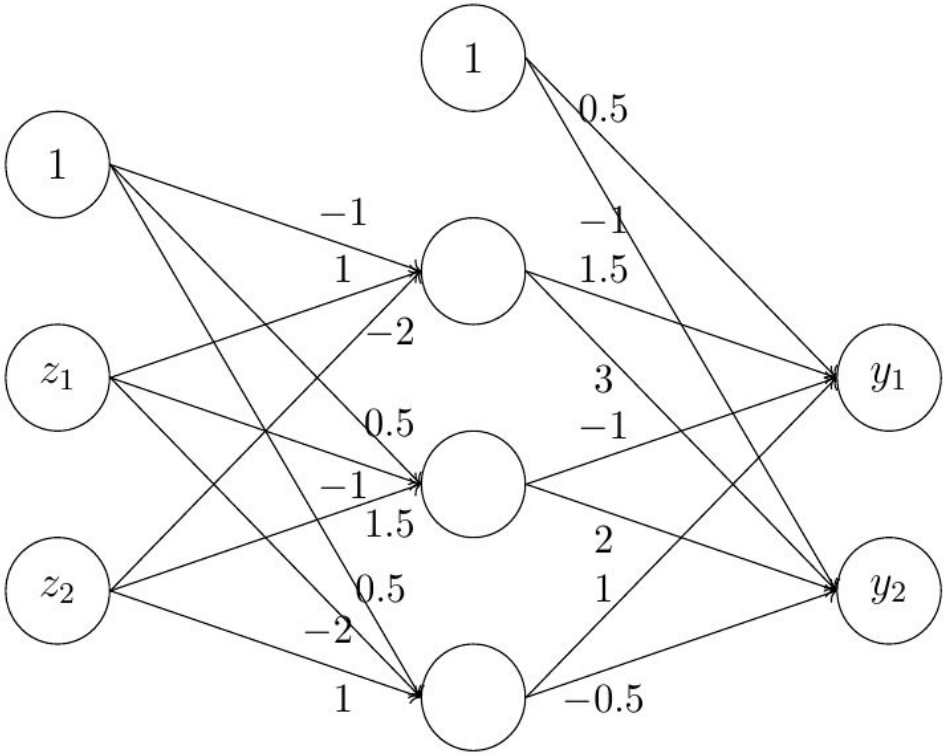
- ➔ Pre-processing: inputs are usually centered in 0 and normalized
- ➔ Weights should not be all 0, but they should be close
- ➔ Weights can be initialized with a normal distribution or a uniform distribution in [-1,1]
- ➔ Glorot initialization: normally distributed weights are scaled by the number of connections of the neuron
- ➔ Stopping condition: absolute error, error improvement, max iterations
- ➔ Multiple initializations are good to avoid local minima (K-fold)

Part 1:
Neural networks



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Exercise: forward propagation

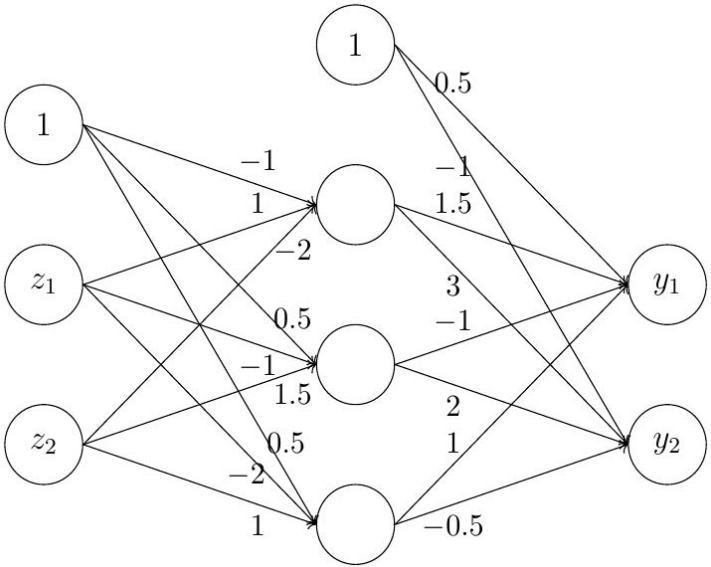


Input: [-1, 1]

Activation function: ReLU

$$\mathbf{z}' = \begin{bmatrix} 1 & \mathbf{z} \end{bmatrix} = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix}$$

Exercise: forward propagation

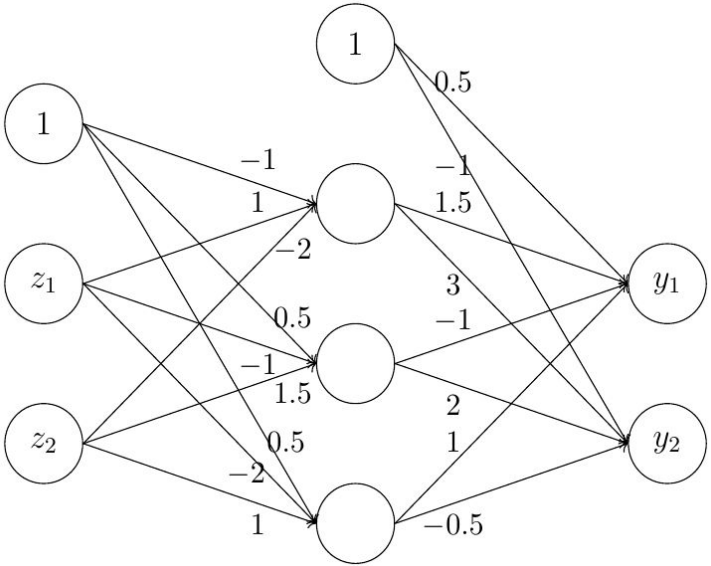


$$\mathbf{a}^{(1)} = \mathbf{z}\mathbf{w}^{(1)} = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} -1 & 0.5 & 0.5 \\ 1 & -1 & -2 \\ -2 & 1.5 & 1 \end{bmatrix}$$

$$\mathbf{a}^{(1)} = \begin{bmatrix} -1-1-2 & 0.5+1+1.5 & 0.5+2+1 \end{bmatrix} = \begin{bmatrix} -4 & 4 & 3.5 \end{bmatrix}$$

$$\mathbf{o}^{(1)} = \sigma(\mathbf{a}^{(1)}) = \begin{bmatrix} 0 & 4 & 3.5 \end{bmatrix}$$

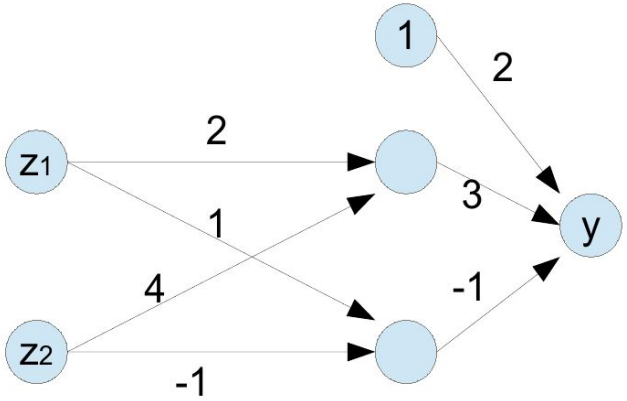
Exercise: forward propagation



$$\mathbf{y} = \begin{bmatrix} 1 & \mathbf{o}^{(1)} \end{bmatrix} \mathbf{w}^{(2)} = \begin{bmatrix} 1 & 0 & 4 & 3.5 \end{bmatrix} \begin{bmatrix} 0.5 & -1 \\ 1.5 & 3 \\ -1 & 2 \\ 1 & 0.5 \end{bmatrix}$$

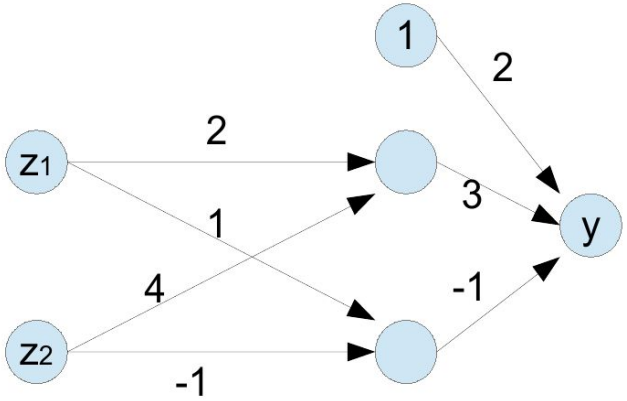
$$\mathbf{y} = \begin{bmatrix} 0.5+0-4+3.5 & -1+0+8+1.75 \end{bmatrix} = \begin{bmatrix} 0 & 8.75 \end{bmatrix}$$

Exercise 3.1



Let us consider a ReLU activation, and an input z=(-2, 3). What is the output?

First layer

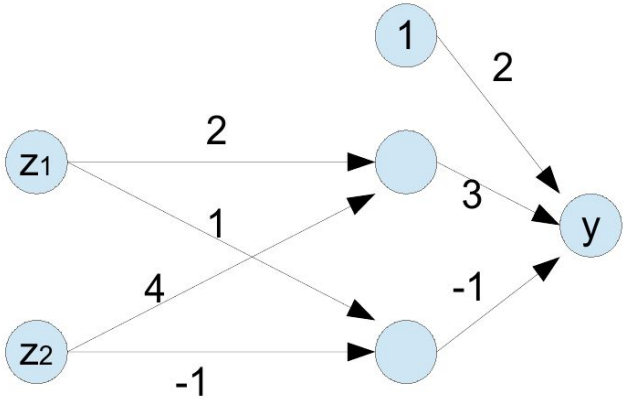


$$\mathbf{w}^{(1)} = \begin{pmatrix} 2 & 1 \\ 4 & -1 \end{pmatrix}$$

$$\mathbf{a}^{(1)} = \mathbf{z}\mathbf{w}^{(1)} = (-2\ 3)\begin{pmatrix} 2 & 1 \\ 4 & -1 \end{pmatrix} = (-4+12\ \ -2-3) = (8\ \ -5)$$

We can then apply the ReLU activation, and get $o^{(1)}$=(8,0)

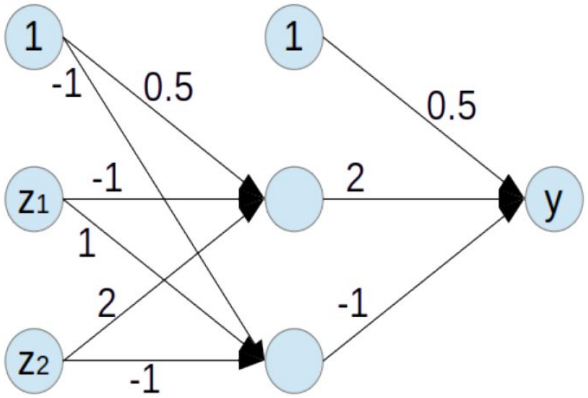
Second layer



$$\mathbf{w}^{(2)} = \begin{pmatrix} 2 \\ 3 \\ -1 \end{pmatrix}$$

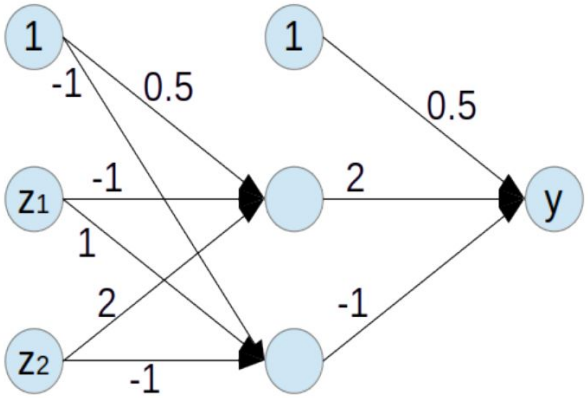
We do not have an activation function on the output, so we get:
$o'^{(1)}w^{(2)}=(1, 8, 0)(2, 3, -1)'=2+24=26$

Exercise 3.2



Let us consider a tanh activation, and an input z=(2, 1). What is the output?

First layer

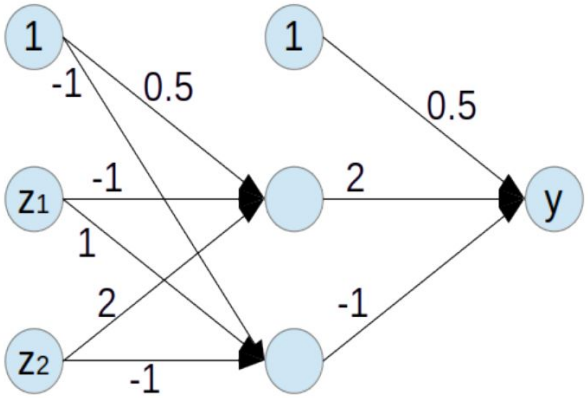


$$\mathbf{w}^{(1)} = \begin{pmatrix} 0.5 & -1 \\ -1 & 1 \\ 2 & -1 \end{pmatrix}$$

$$\mathbf{a}^{(1)} = \mathbf{z}\mathbf{w}^{(1)} = (1\ 2\ 1) \begin{pmatrix} 0.5 & -1 \\ -1 & 1 \\ 2 & -1 \end{pmatrix} = (0.5 - 2 + 2 \ \ -1 + 2 - 1) = (0.5\ 0)$$

We have tanh(0.5)=0.462 and tanh(0)=0, so o'$^{(1)}$=(0.462, 0)

Second layer



$$\mathbf{w}^{(2)} = \begin{pmatrix} 0.5 \\ 2 \\ -1 \end{pmatrix}$$

We do not have an activation function on the output, so we get:
$o'^{(1)}w^{(2)}=(1, 0.462, 0)(0.5, 2, -1)'=0.5+0.924+0=1.424$